The exact Riemannian Hessian of a T3 fit and the two-hole sweep

T3Toolbox notes — N. Alger, B. Christierson & collaborators

October 28, 2026

Preliminary version (slice S15a). Every code reference in this note is a real name at commit `f739085` (the end of the exact-Hessian arc, release 2026.3.0, unreleased). The module layout is about to be reorganized; the references are re-checked, and renamed where needed, in the final pass (S15b) before the release. Equation and algorithm numbers of the T4S paper are those the code's docstrings cite (the local `t4s.pdf`, newer than arXiv); they too are re-verified in S15b.

Abstract

`newton_cg(..., hessian='exact')` solves each Newton step of a weighted least-squares fit on the fixed-rank Tucker tensor train (T3) manifold with the *exact* Riemannian Hessian: the Gauss–Newton term plus the Weingarten term, which pairs the normal part of the residual gradient with the manifold's second fundamental form. We write that term as a core-coordinate second derivative of the multilinear parametrization Φ, pulled back by a *lift* from gauged frame coordinates to raw core directions, and compute the second derivative by a *two-hole sweep*: the directional derivative of the library's corewise transpose along a core direction, written as the product rule of the Gauss–Newton scan bodies it differentiates. The note records the formulas as implemented — the lift and its damping, the dotted forward and adjoint sweeps with two-index jets, the factored assembly, the T3-to-T3 term, the uniform masking rule, the tied lift of shared factors, the canonical zero-point frame of the Newton reference, and the Steihaug–Toint trust region — with an algorithm box for each sweep naming the function that implements it. Every displayed identity was checked against the library (Section 7).

1 Setup and notation

The format. A T3 with raw cores $\theta = \big((U_i)_{i=1}^d, (G_i)_{i=1}^d\big)$, $U_i \in \mathbb{R}^{n_i \times N_i}$, $G_i \in \mathbb{R}^{r_{i-1} \times n_i \times r_i}$, $r_0 = r_d = 1$, represents $X = \Phi(\theta) = \mathrm{TT}(G_1, \dots, G_d) \times_1 U_1 \cdots \times_d U_d$ (code: `TuckerTensorTrain.data` = (tucker_cores, tt_cores); modes are 1-based here and 0-based in the code). Φ is *multilinear* in its $2d$ cores, so its derivatives are core replacements:

$$D\Phi(\theta)[\hat\theta] = \sum_a \Phi(\theta;\, a \leftarrow \hat\theta_a), \qquad D^2\Phi(\theta)[\hat\theta, \hat\theta'] = \sum_{a \neq b} \Phi(\theta;\, a \leftarrow \hat\theta_a,\, b \leftarrow \hat\theta'_b), \tag{1}$$

sums over the $2d$ core slots a, b (single and double replacement); the diagonal blocks $a = b$ vanish.

The frame. At a point X the orthogonal frame (T4S [1] App. A.1, A.3, Algorithm 11; built by `t3_orthogonal_representations`, which sweeps left-then-right where Algorithm 11 sweeps right-then-left — the representations are equivalent, and everything below is recomputed from the stored cores) is (U_i, O_i, P_i, Q_i) = `T3Frame.data` = (up, down, left, right): $U_i \in \mathbb{R}^{n_i \times N_i}$ with orthonormal rows, left-orthogonal $P_i \in \mathbb{R}^{r^L_{i-1} \times n_i \times r^L_i}$, right-orthogonal $Q_i \in \mathbb{R}^{r^R_{i-1} \times n_i \times r^R_i}$, and $O_i \in \mathbb{R}^{r^L_{i-1} \times n^D_i \times r^R_i}$

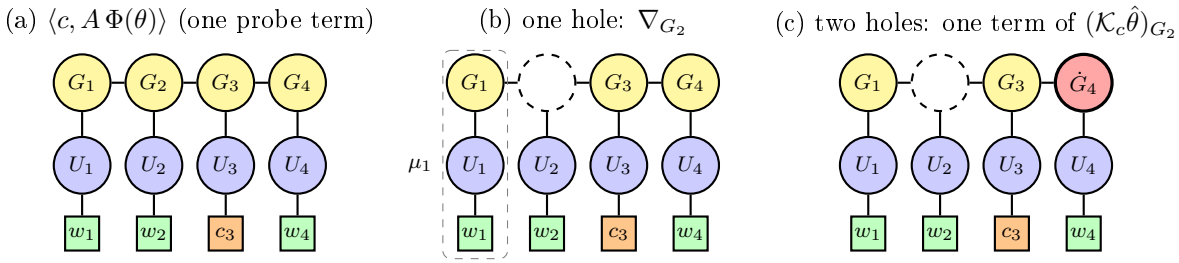


Figure 1: Cores as nodes, bonds as edges (TT cores on top, Tucker factors below, sample vectors as squares). (a) One term of $\langle c, A\Phi(\theta)\rangle$ for a probe: the weighted residual c_3 sits on the free mode. (b) The one-hole sweep (the corewise transpose) leaves one core out: the open legs are its gradient block, here the left environment μ_1 (dashed box), the projected probe $\xi_2 = U_2 w_2$ and a right environment. (c) The two-hole sweep leaves one core out *and* replaces another by its direction ($\dot{G}_4$, red); summing over the replaced core $b \neq a$ gives block a of $\mathcal{K}_c\hat{\theta}$ (eq. (5)).

orthonormal over its middle index ($n_i^D \le n_i$; $n_i^D < n_i$ is *Tucker slack*, $r_i^R < r_i^L$ *TT slack*). The base point in its left-canonical gauge is $\theta = (U, P)$ (`ManifoldGeometryOps.base_point`). A tangent vector has gauged coordinates $v = (\delta U_i, \delta G_i)$, $\delta U_i \in \mathbb{R}^{n_i^D \times N_i}$, $\delta G_i \in \mathbb{R}^{r_{i-1}^L \times n_i \times r_i^R}$ (code: `tucker_variations` V, `tt_variations` H), and represents (T4S eq. (47); `tv_to_dense`)

$$\mathcal{E}v = \sum_i \left[P_{<i}\,\delta G_i\,Q_{>i}\right] \times_j U_j + \sum_i \left[P_{<i}\,O_i\,Q_{>i}\right] \times_i \delta U_i \times_{j\neq i} U_j, \tag{2}$$

where $P_{<i} = P_1 \cdots P_{i-1}$ and $Q_{>i} = Q_{i+1} \cdots Q_d$ are TT chains, and $C \times_k M$ contracts index k of a core C with the *rows* of a matrix M (so $\Phi(\theta) = \mathrm{TT}(G) \times_1 U_1 \cdots$). The gauge conditions $\delta U_i U_i^{\mathsf{T}} = 0$ and $(P_i^{\langle 2\rangle})^{\mathsf{T}} \delta G_i^{\langle 2\rangle} = 0$ ($i < d$; T4S (48)–(49)) are imposed by the coordinate-orthogonal projection Π (`tv_orthogonal_gauge_projection`); on gauged coordinates $\mathcal{E}$ is an isometry, so the coordinate dot product *is* the Hilbert–Schmidt metric. The tangent *sampling Jacobian* is $\mathcal{J} = A \circ \mathcal{E}$ (`SamplingKind.forward`, T4S Algorithms 6–7), its transpose $\mathcal{J}^{\mathsf{T}}$ (`SamplingKind.transpose`, Algorithm 8).

The objective. For a linear sampling operator A (entries, apply, probe or their symmetric derivative jets; all six kinds are linear in the tensor), data y and a residual weight ω,

$$f(X) = \tfrac{1}{2}\|\omega \odot r\|^2, \quad r = AX - y, \qquad \nabla f = A^{\mathsf{T}}(\omega^2 \odot r), \qquad \nabla^2 f = A^{\mathsf{T}}\omega^2 A, \tag{3}$$

$\mathrm{grad}\, f = P_X \nabla f$ ($P_X = \mathcal{E}\Pi\mathcal{E}^{\mathsf{T}}$, coordinates $g = \Pi\mathcal{J}^{\mathsf{T}}(\omega^2 \odot r)$, the model's `misfit_gradient`), and the *normal part* $N = (I - P_X)\nabla f$. The jets of the derivative kinds carry an order axis $t = 0, \ldots, m$, $m =$ `order` (`docs/symmetric_probe_derivatives.tex`). Batch blocks follow the library: `W` the sample stack, `K` the tangent stack (directions at one point), `C` the frame stack, frame-inner `W` + `K` + `C`; the curvature adds no new block.

2 The exact Riemannian Hessian

2.1 The Weingarten term as a second derivative of Φ

Proposition 1 (the curvature term, [2])**.** *Let $\zeta, \zeta' \in T_X\mathcal{M}$ and let $\hat{\theta}, \hat{\theta}'$ be any raw directions with $D\Phi(\theta)[\hat{\theta}] = \zeta$, $D\Phi(\theta)[\hat{\theta}'] = \zeta'$. Then*

$$\langle \mathrm{Hess}\, f(X)[\zeta],\, \zeta'\rangle = \langle \omega \odot A\zeta,\, \omega \odot A\zeta'\rangle + \langle N,\, D^2\Phi(\theta)[\hat{\theta}, \hat{\theta}']\rangle, \tag{4}$$

and the second term does not depend on the choice of lifts.

Proof. For an embedded submanifold, $\operatorname{Hess} f(X)[\zeta] = P_X \nabla^2 f\, \zeta + \mathfrak{W}_X(\zeta, N)$ with the Weingarten map $\mathfrak{W}_X(\zeta, N) = P_X\,(D_\zeta P)\,N$, and $\langle \mathfrak{W}_X(\zeta, N),\, \zeta' \rangle = \langle N,\, \mathrm{II}_X(\zeta, \zeta') \rangle$ [2]. The curve $s \mapsto \Phi(\theta + s\hat{\theta} + s'\hat{\theta}')$ lies in $\mathcal{M}$; by the Gauss formula the normal part of its mixed second derivative is $\mathrm{II}_X(\zeta, \zeta')$, so $\langle N,\, D^2\Phi[\hat{\theta}, \hat{\theta}'] \rangle = \langle N,\, \mathrm{II}_X(\zeta, \zeta') \rangle$, which depends on ζ, ζ' only. The first term is $\langle \zeta',\, P_X A^{\mathsf{T}} \omega^2 A \zeta \rangle = \langle \omega A \zeta,\, \omega A \zeta' \rangle$ since ζ' is tangent. □

Lift independence is visible directly for the gauge directions: a *vertical* direction $\hat{\theta}_v = V\theta$, with V a linear generator of the gauge action ($D\Phi(\theta)[V\theta] = 0$ for *every* θ), satisfies, after differentiating that identity along $\hat{\theta}'$, $D^2\Phi[\hat{\theta}_v, \hat{\theta}'] = -D\Phi[V\hat{\theta}'] \in T_X\mathcal{M}$, which N annihilates. (At a slack frame $\ker D\Phi(\theta)$ is larger than the gauge orbit's tangent; Proposition 1 covers every lift.) No derivative of the frame (no SVD perturbation) enters — this is the reason for the route (the archived design plan's "route (b)"; contrast Novikov, Rakhuba & Oseledets [4], whose stop-gradient on the projector gives exactly the Gauss–Newton term).

Core-coordinate second derivatives. For a fixed ambient Z define

$$\mathcal{K}_Z\, \hat{\theta} \;=\; \frac{d}{ds} \nabla_\theta \langle Z,\, \Phi(\theta + s\hat{\theta}) \rangle \Big|_{s=0}, \qquad \langle \hat{\theta}',\, \mathcal{K}_Z \hat{\theta} \rangle = \langle Z,\, D^2\Phi(\theta)[\hat{\theta}, \hat{\theta}'] \rangle, \tag{5}$$

symmetric and linear in Z. Block a of $\mathcal{K}_Z\hat{\theta}$ is $\sum_{b \neq a} \nabla_{\theta_a} \langle Z,\, \Phi(\theta;\, b \leftarrow \hat{\theta}_b) \rangle$ — one core out, one replaced (Figure 1c). Two instances are needed, and $\langle N,\, D^2\Phi \rangle$ is their difference:

- $Z = \nabla f = A^{\mathsf{T}}(\omega^2 \odot r)$: $\nabla_\theta \langle Z,\, \Phi(\theta) \rangle$ is the kind's *corewise transpose* at the weighted residual $c = \omega^2 \odot r$, so $\mathcal{K}_{A^{\mathsf{T}}\omega^2 r}$ is its directional derivative, the *two-hole sweep* of Section 3 (`SamplingKind.corewise_hessian`);
- $Z = \operatorname{grad} f$, a doubled-rank T3 ($\texttt{tv_to_t3}(\text{frame}, g)$; T4S App. A.3.1, (50)–(53)): the T3-to-T3 two-hole of Section 3.5 (`ManifoldGeometryOps.tangential_curvature`).

Writing $\mathcal{K}_N := \mathcal{K}_{A^{\mathsf{T}}\omega^2 r} - \mathcal{K}_{\operatorname{grad} f}$, (4) reads $\langle \operatorname{Hess} f[\zeta],\, \zeta' \rangle = \langle \omega A \zeta,\, \omega A \zeta' \rangle + \langle \hat{\theta}',\, \mathcal{K}_N \hat{\theta} \rangle$.

2.2 The Lagrangian form, and the two levels

Equivalently, (4) is the core-coordinate Hessian of the *Lagrangian*

$$\Lambda(\theta) = f(\Phi(\theta)) - \langle \operatorname{grad} f(X),\, \Phi(\theta) \rangle, \qquad \nabla_\theta \Lambda = D\Phi^{\mathsf{T}} N = 0, \qquad \nabla^2_\theta \Lambda = D\Phi^{\mathsf{T}}\, \operatorname{Hess} f(X)\, D\Phi, \tag{6}$$

($\operatorname{grad} f(X)$ frozen). θ is a critical point of Λ, so its Hessian is tensorial and the vertical directions lie in its null space. Dropping the gradient term gives the plain core Hessian of $f \circ \Phi$:

level	operator on gauged coordinates	exact when
0, Gauss–Newton	$\Pi \mathcal{J}^{\mathsf{T}} \omega^2 \mathcal{J} \Pi$	$N = 0$
1, core-space Newton	$\Pi \mathcal{J}^{\mathsf{T}} \omega^2 \mathcal{J} \Pi + \Pi \mathcal{L}^{\mathsf{T}} \mathcal{K}_{A^{\mathsf{T}}\omega^2 r} \mathcal{L} \Pi = \Pi \mathcal{L}^{\mathsf{T}} \nabla^2_\theta (f \circ \Phi) \mathcal{L} \Pi$	$\operatorname{grad} f = 0$
2, Riemannian Newton (shipped)	level 1 $- \Pi \mathcal{L}^{\mathsf{T}} \mathcal{K}_{\operatorname{grad} f} \mathcal{L} \Pi = \Pi \mathcal{L}^{\mathsf{T}} \nabla^2_\theta \Lambda \mathcal{L} \Pi$	always

($\mathcal{L}$ is the undamped lift of Section 2.3; with a damping $\varepsilon > 0$ the Gauss–Newton term stays exact and only the curvature term is filtered.) Level 1 is the second derivative of f along the parametrization chart $t \mapsto \Phi(\theta + t\mathcal{L}p)$, which is only a first-order retraction; it differs from level 2 by $\langle \operatorname{grad} f,\, P_X D^2\Phi[\cdot, \cdot] \rangle = O(\|\operatorname{grad} f\|)$ and depends on the lift. Level 2 is what the library ships; its correction $\mathcal{K}_{\operatorname{grad} f}$ is independent of the number of samples.

The retraction is second order (measured). The model is consistent with the step the optimizer takes because the T3-SVD retraction (`tv_retract`, the implicit T3-SVD of T4S Algorithm 10, App. A.2) is *second order*: $\frac{d^2}{dt^2} f(R_X(t\zeta))|_0 = \langle \mathrm{Hess}\, f[\zeta], \zeta \rangle$. The archived design note argues it as a composition of exact metric projections onto smooth bounded-rank varieties (each truncation is Eckart–Young for one unfolding behind an isometric environment; cf. [6]); the library does not rely on the argument but on measurement: 252.85288420 vs 252.85288182 (design-plan check F6), the `Problem.local_model` doctest, the per-kind test within 10^{-3} of the curvature term itself, the shared retraction's finite-difference ratios 4.0 and 4.25 ($O(h^2)$, F14), and 3.7×10^{-6} here (check C8a, Richardson). The second order presumes a minimal, numerically full-rank frame (`retract` preserves the frame ranks only there — a caveat, not a precondition).

2.3 The lift

Frame factors. The frame construction produces, and discards, two small matrices per mode; the library recomputes them by contraction only (`fv_center_factors`; I-1: never stored on the frame, never re-SVD'd):

$$T_i = \langle P_{>i}, Q_{>i} \rangle \in \mathbb{R}^{r_i^L \times r_i^R} \ (i < d), \qquad H_i = P_i \times_3 T_i \ (H_d = P_d), \qquad S_i^{\mathsf{T}} = \langle O_i, H_i \rangle_{\{1,3\}} \in \mathbb{R}^{n_i^D \times n_i}, \tag{7}$$

(T_i contracts the cores right of bond i: the right-to-left zipper of the left chain against the right chain, `tt_zipper_right_to_left`). Then $H_i = O_i \times_2 S_i^{\mathsf{T}}$, the centers are the point's own core with the orthogonality center at i ($X = P_{<i} H_i Q_{>i} \times U$), $P_{>i} = T_i\, Q_{>i}$ (the left-canonical tail is T_i times the right-orthogonal one), $\sigma(S_i)$ is the mode-i Tucker spectrum, $\sigma(T_i)$ the bond-i TT spectrum, and $\|X\|_{HS}^2 = \|S_1^{\mathsf{T}}\|_F^2$ (checks C1a–e).

Proposition 2 (the lift). *Define $\mathcal{L}v = \hat{\theta} = (\dot{U}_i, \dot{G}_i)$, a variation of the base point $\theta = (U, P)$, by*

$$\dot{U}_i = (S_i^{\mathsf{T}})^+ \, \delta U_i, \qquad \dot{G}_i = \delta G_i \times_3 T_i^+ \ (i < d), \qquad \dot{G}_d = \delta G_d. \tag{8}$$

If every S_i^{T} has full row rank and every T_i full column rank, then $D\Phi(\theta)[\mathcal{L}v] = \mathcal{E}v$ for all *coordinates v, gauged or not.*

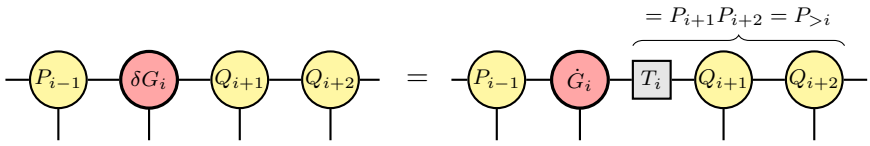
Proof. TT slot: $P_{<i}\dot{G}_i P_{>i} = P_{<i}\,(\dot{G}_i T_i)\, Q_{>i} = P_{<i}\,\delta G_i T_i^+ T_i\, Q_{>i}$, and $T_i^+ T_i = I$ for full column rank. Tucker slot: replacing U_i by $\dot{U}_i$ in $X = P_{<i} H_i Q_{>i} \times U$ gives $P_{<i}(H_i \times_2 \dot{U}_i) Q_{>i} = P_{<i}\big(O_i \times_2 (S_i^{\mathsf{T}} \dot{U}_i)\big) Q_{>i}$, and $S_i^{\mathsf{T}} (S_i^{\mathsf{T}})^+ = I$ for full row rank. Compare (2). □

Every frame `t3_orthogonal_representations` builds meets the hypothesis when it is numerically full rank, minimal or not: Tucker slack makes S_i^{T} wide and TT slack makes T_i tall, and on both the pseudo-inverse is an exact one-sided inverse (checks C2a–c, F2/F3). Only a hand-built frame with a wide T_i would give a least-squares lift (a caveat the library never reaches). Figure 2 draws the two slots.

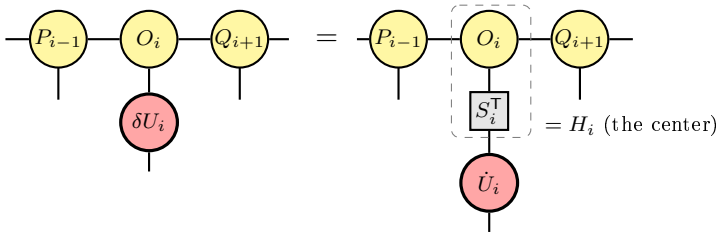
Damping. The lift is the only place $1/\sigma$ appears (the TT part is the analogue of Psenka & Boumal's R_k^{-1} [3]), so it is damped by the Tikhonov filter

$$f_\varepsilon(\sigma) = \frac{\sigma}{\sigma^2 + \varepsilon^2} \ \ (0 \text{ for } \sigma \le \mathrm{rcond} \cdot \sigma_1), \qquad M^{+\varepsilon} = V f_\varepsilon(\Sigma) U^{\mathsf{T}} = (M^{\mathsf{T}} M + \varepsilon^2 I)^{-1} M^{\mathsf{T}}, \tag{9}$$

with the absolute damping $\varepsilon =$ `curvature_damping` $\cdot \|X\|_{HS}$ (`filtered_reciprocal`, `damped_pseudo_inverse` in `backend/linalg.py`; $\|X\|_{HS} = \|S_1^{\mathsf{T}}\|_F$; default `DEFAULT_CURVATURE_DAMPING`



(a) TT slot: $\dot{G}_i = \delta G_i \times_3 T_i^{+\varepsilon}$, exact when $T_i^+ T_i = I$ (TT slack: T_i tall)



(b) Tucker slot: $\dot{U}_i = (S_i^\mathsf{T})^{+\varepsilon} \delta U_i$, exact when $S_i^\mathsf{T}(S_i^\mathsf{T})^+ = I$ (Tucker slack: S_i^T wide)

Figure 2: The lift, slot by slot (red: the variation). (a) The frame's TT slot $P_{<i}\,\delta G_i\,Q_{>i}$ equals the raw slot $P_{<i}\,\dot{G}_i\,P_{>i}$ because the left-canonical tail is $P_{>i} = T_i Q_{>i}$. (b) The frame's Tucker slot pairs δU_i with the orthonormal O_i; the raw slot pairs $\dot{U}_i$ with the center $H_i = O_i \times_2 S_i^\mathsf{T}$. The pseudo-inverses are the only place an inverse singular value enters the Hessian.

$= 10^{-6}$ in `backend/geometry.py`). $f_\varepsilon \le 1/(2\varepsilon)$ (check C4a), $f_\varepsilon \approx 1/\sigma$ for $\sigma \gg \varepsilon$, $\approx \sigma/\varepsilon^2$ below; the relative clip (default dtype-ε times the larger dimension, `numpy.linalg.pinv`'s) maps roundoff singular values to exactly 0, which is why a zero-padded rank-continuation restart (new singular values $\sim 10^{-17}$) needs no special case: there the lift satisfies $D\Phi[\mathcal{L}v] = \mathcal{E}(Q_c v)$ with Q_c the projection off the clipped directions, the Hessian is Gauss–Newton in them, and the trust region bounds the step. The last TT slot passes through undamped. The damping is a *solver option*, never a geometry field: it reaches the computation only through the traced leaf `LiftFactors.eps`, so a new damping never recompiles (I-22).

The transpose. $\mathcal{L}^\mathsf{T} y$ maps raw core variations $y = (Y_i^U, Y_i^G)$ to *ungauged* coordinates, $\delta U_i = ((S_i^\mathsf{T})^{+\varepsilon})^\mathsf{T} Y_i^U$, $\delta G_i = Y_i^G \times_3 (T_i^{+\varepsilon})^\mathsf{T}$, $\delta G_d = Y_d^G$, with the *same* factors, so $\mathcal{L}_\varepsilon^\mathsf{T} \mathcal{K} \mathcal{L}_\varepsilon$ is symmetric for every ε (check C3). The Hessian applies Π outside.

2.4 The Newton system

In gauged coordinates, with the identity regularizer $\rho(X) = \frac{\lambda}{2}\|X\|^2$ (whose Hessian $\lambda\Pi$ is already exact: X lies in its own tangent space),

$$\boxed{\mathcal{H}p \;=\; \Pi\mathcal{J}^\mathsf{T}\omega^2\mathcal{J}\Pi p \;+\; \Pi\mathcal{L}^\mathsf{T}\big(\mathcal{K}_{A^\mathsf{T}\omega^2 r} - \mathcal{K}_{\operatorname{grad} f}\big)\mathcal{L}\Pi p \;+\; \lambda\Pi p} \tag{10}$$

(`LocalModel.hvp`); the curvature uses the *misfit* gradient only. On the corewise geometry the exact Hessian is the plain Hessian of $f \circ \Phi$, $\mathcal{J}_c^\mathsf{T}\omega^2\mathcal{J}_c + \mathcal{K}_{A^\mathsf{T}\omega^2 r}$: no lift, no gradient term (the corewise geometry's `lift` is the identity, `tangential_curvature` returns `None`). The model does *one* misfit transpose per Newton step: `Problem.local_model(hessian='exact')` computes $g = \Pi\mathcal{J}^\mathsf{T}\omega^2 r$ once into `CurvatureState.misfit_gradient`, read by both `LocalModel.gradient` (bit-identical to Gauss–Newton's, check C14c) and `curvature_precompute`. The per-step overhead is the doubled-rank $Z = \operatorname{grad} f$, one T3-to-T3 sweep and $2d-1$ small SVDs.

Algorithm 1 The lift $\hat{\theta} = \mathcal{L}_\varepsilon v$ and its adjoint $\mathcal{L}_\varepsilon^{\mathsf{T}}$ (per-frame factors once, per matvec two contractions per mode)

Require: orthogonal frame (U, O, P, Q); relative damping $c \geq 0$; coordinates $v = (\delta U_i, \delta G_i)$ (stack $\mathtt{K} + \mathtt{C}$)

Ensure: raw directions $\hat{\theta} = (\dot{U}_i, \dot{G}_i)$ of $\theta = (U, P)$

$T_i \leftarrow \langle P_{>i},\, Q_{>i} \rangle$ (zipper); $H_i \leftarrow P_i \times_3 T_i$; $S_i^{\mathsf{T}} \leftarrow \langle O_i,\, H_i \rangle_{\{1,3\}}$ ▷ `fv_center_factors`

$\varepsilon \leftarrow c\, \|S_1^{\mathsf{T}}\|_F$; $(S_i^{\mathsf{T}})^{+\varepsilon},\ T_i^{+\varepsilon} \leftarrow$ one thin SVD each, filter (9) ▷ `fv_lift_factors`

for $i = 1, \ldots, d$ **do** ▷ `tv_corewise_lift`

  $\dot{U}_i \leftarrow (S_i^{\mathsf{T}})^{+\varepsilon}\, \delta U_i$ ▷ `'Cux,KCxn->KCun'`

  $\dot{G}_i \leftarrow \delta G_i \times_3 T_i^{+\varepsilon}$ if $i < d$, else $\dot{G}_d \leftarrow \delta G_d$ ▷ `'KCaib,Cbc->KCaic'`

end for

adjoint: $\delta U_i \leftarrow ((S_i^{\mathsf{T}})^{+\varepsilon})^{\mathsf{T}} Y_i^U$, $\delta G_i \leftarrow Y_i^G \times_3 (T_i^{+\varepsilon})^{\mathsf{T}}$ ▷ `tv_corewise_lift_transpose`

Code: `fv_operations.fv_center_factors`, `fv_operations.fv_lift_factors` (`LiftFactors`), `linalg.damped_pseudo_inverse`, `linalg.filtered_reciprocal`, `tv_operations.tv_corewise_lift[_transpose]`; uniform: `ufv_operations.ufv_lift_factors`, `utv_operations.utv_corewise_lift[_transpose]`.

Where the term vanishes. At a slot of *maximal* rank (a Tucker rank equal to the mode size, a TT rank equal to the product of its neighbours) the representable set is locally open in that slot's directions and N has no component there; at $d = 1$ every slot is maximal and $N \equiv 0$. The formula needs no special case (the filtered pseudo-inverses and the two-hole sum give $\mathcal{L}^{\mathsf{T}} \mathcal{K}_N \mathcal{L} = 0$); the manifold geometry's `curvature_precompute` returns `None` at $d = 1$ purely to skip the work.

3 The two-hole sweep

3.1 The one-hole sweep it differentiates

At the *corewise frame* (U, G, G, G) of raw cores $\theta = (U, G)$ (`t3_corewise_frame`) the tangent transpose of T4S Algorithm 8 is the core gradient $\nabla_\theta \langle c,\, A\Phi(\theta) \rangle$ (T4S §6.3, the corewise substitution $P, Q, O \to G$; `t3_probe_corewise_transpose`). With edge variables on bonds $0, \ldots, d$ (core i joins bonds $i-1$ and i; $G_i(x)$ contracts the middle index of G_i with x):

$$\xi_i = U_i w_i, \quad \mu_i = \mu_{i-1} G_i(\xi_i),\ \mu_0 = 1, \quad \nu_{i-1} = G_i(\xi_i)\nu_i,\ \nu_d = 1, \quad \eta_i = \mu_{i-1} G_i \nu_i, \tag{11}$$

$$\delta\tilde{\eta}_i = U_i c_i, \quad \tilde{\tau}_i = \tilde{\tau}_{i-1} G_i(\xi_i) + \mu_{i-1} G_i(\delta\tilde{\eta}_i),\ \tilde{\tau}_0 = 0, \quad \tilde{\sigma}_{i-1} = G_i(\xi_i)\tilde{\sigma}_i + G_i(\delta\tilde{\eta}_i)\nu_i,\ \tilde{\sigma}_d = 0, \tag{12}$$

$$\delta\tilde{\xi}_i = \tilde{\tau}_{i-1} G_i \nu_i + \mu_{i-1} G_i \tilde{\sigma}_i, \tag{13}$$

$$\delta\tilde{U}_i = \textstyle\sum_{\mathtt{W}} \big(\eta_i \otimes c_i + \delta\tilde{\xi}_i \otimes w_i\big), \qquad \delta\tilde{G}_i = \textstyle\sum_{\mathtt{W}} \big(\mu_{i-1} \otimes \xi_i \otimes \tilde{\sigma}_i + \tilde{\tau}_{i-1} \otimes \xi_i \otimes \nu_i + \mu_{i-1} \otimes \delta\tilde{\eta}_i \otimes \nu_i\big). \tag{14}$$

The TT gradient has three trilinear groups (the core-adjoints of the forward σ, τ, $\delta\eta$ of T4S Algorithm 7); the Tucker gradient has one internal edge. Apply and entries (`t3_apply_corewise_transpose`) are the all-modes case: a scalar residual c seeds one propagation sweep $\hat{\sigma}_{i-1} = G_i(\xi_i)\hat{\sigma}_i$, $\hat{\sigma}_d = c$, and $\delta\tilde{G}_i = \sum_{\mathtt{W}} \mu_{i-1} \otimes \xi_i \otimes \hat{\sigma}_i$, $\delta\tilde{U}_i = \sum_{\mathtt{W}} \delta\hat{\xi}_i \otimes w_i$ with $\delta\hat{\xi}_i = \mu_{i-1} G_i \hat{\sigma}_i$.

3.2 Two holes: the product rule of the one-hole sweep

Every quantity in (11)–(14) is multilinear in the cores and the sample vectors. For fixed c the one-hole output is $\nabla_\theta \langle c,\, A\Phi(\theta) \rangle$, so by (5) $\mathcal{K}_c \hat{\theta}$ is its directional derivative along $\hat{\theta} = (\dot{U}_i, \dot{G}_i)$, i.e.

Algorithm 2 The exact Hessian-vector product (10)

Require: local model at X: frame, sampling sweep, residual r, curvature data ($\mathcal{L}$ factors, $Z = \operatorname{grad} f$, Z-sweep); direction p (stack $\mathtt{K}+\mathtt{C}$)
Ensure: $\mathcal{H}p$

$p \leftarrow \Pi p$; $\quad z \leftarrow \mathcal{J}p$ ▷ `SamplingKind.forward`
$\hat{\theta} \leftarrow \mathcal{L}p$ ▷ `geom.lift`
$k \leftarrow \mathcal{K}_{A^{\mathsf{T}}\omega^2 r}\,\hat{\theta}$ ▷ two-hole, `kind.corewise_hessian` (Alg. 6)
$k \leftarrow k - \mathcal{K}_{\operatorname{grad} f}\,\hat{\theta}$ ▷ T3–T3 two-hole, `geom.tangential_curvature` (Alg. 7)
$y \leftarrow \mathcal{L}^{\mathsf{T}}k$ ▷ `geom.lift_transpose`, ungauged
return $\Pi\big(\mathcal{J}^{\mathsf{T}}\omega^2 z + y\big) + \lambda\Pi p$ ▷ one projection for both terms

Code: `optimizers.LocalModel.hvp`, `LocalModel._curvature_block`, `curvature_hvp`, `quadratic` = `gn_quadratic` + `curvature_quadratic`; per frame: `Problem.local_model`, `CurvatureState`, `ManifoldGeometryOps.curvature_precompute` (`ManifoldCurvatureData`: `fv_lift_factors`, `tv_to_t3`, `t3_precompute_inner_product_sweep`, `_lift_sigma_min`); uniform: `UniformManifoldGeometryOps.curvature_precompute`.

Algorithm 3 The one-hole sweep (reference): the corewise probe transpose $\nabla_\theta\langle c,\, A\Phi(\theta)\rangle$

Require: $\theta = (U_i, G_i)$; probe vectors w_i (stack $\mathtt{W}$); weighted residuals $c_i = \omega_i^2 \odot r_i$
Ensure: $(\delta\tilde{U}_i, \delta\tilde{G}_i)$, $\mathtt{W}$ summed

$\xi_i, \mu_i, \nu_i, \eta_i$ by (11) ▷ frame sweep (T4S Alg. 5); `tv_precompute_probe_frame_sweep`
$\delta\tilde{\eta}_i \leftarrow U_i c_i$ ▷ `compute_deta_tilde`
for $i = 1, \dots, d$ **do** $\tilde{\tau}_i \leftarrow \tilde{\tau}_{i-1} G_i(\xi_i) + \mu_{i-1} G_i(\delta\tilde{\eta}_i)$ ▷ `compute_tau_tilde`
end for
for $i = d, \dots, 1$ **do** $\tilde{\sigma}_{i-1} \leftarrow G_i(\xi_i)\tilde{\sigma}_i + G_i(\delta\tilde{\eta}_i)\nu_i$ ▷ `compute_sigma_tilde`
end for
$\delta\tilde{\xi}_i \leftarrow \tilde{\tau}_{i-1} G_i \nu_i + \mu_{i-1} G_i \tilde{\sigma}_i$ ▷ `compute_dxi_tilde`
$\delta\tilde{U}_i,\ \delta\tilde{G}_i$ by (14) ▷ `assemble_tucker_variations`, `assemble_tt_variations`

Code: `probing.t3_probe_corewise_transpose` = `probing.tv_probe_transpose` at `fv_conversions.t3_corewise_frame` (T4S §6.2.3, Algorithm 8; §6.3). Jets: `sampling_derivatives.t3_probe_corewise_derivatives_transpose`, `tv_probe_transpose_derivatives_from_sweep`.

the *product rule* applied to every line of Algorithm 3: each edge variable gains a *dotted* companion carrying the tangent stack $\mathtt{K}$ ("exactly one core replaced"), while c, w and the undotted state carry $\mathtt{W}+\mathtt{C}$ only. Two observations make this cheap:

1. **The dotted forward is the Gauss–Newton tangent forward at the corewise frame.** Differentiating (11) gives $\dot{\xi}_i = \dot{U}_i w_i$, $\dot{\mu}_i = \dot{\mu}_{i-1} G_i(\xi_i) + \mu_{i-1}\dot{G}_i(\xi_i) + \mu_{i-1} G_i(\dot{\xi}_i)$, $\dot{\nu}_{i-1} = G_i(\xi_i)\dot{\nu}_i + \dot{G}_i(\xi_i)\nu_i + G_i(\dot{\xi}_i)\nu_i$, $\dot{\eta}_i = \dot{\mu}_{i-1} G_i \nu_i + \mu_{i-1}\dot{G}_i\nu_i + \mu_{i-1} G_i \dot{\nu}_i$ — exactly T4S Algorithm 7 $(\sigma, \tau, \delta\eta)$ with $Q = O = P = G$ and variation $(\dot{U}, \dot{G})$. The library calls its own GN tangent forward (`compute_sigma`, `compute_tau`, `compute_deta`; jets: `compute_sigma_jets`, `compute_tau_jets`, `compute_deta_jets`).

2. **No undotted η or $\delta\tilde{\xi}$ at the corewise frame is needed.** With c and w fixed, $\frac{d}{ds}\delta\tilde{U}_i = \sum_{\mathtt{W}}(\dot{\eta}_i \otimes c_i + \dot{\delta\tilde{\xi}}_i \otimes w_i)$, and $\delta\tilde{G}_i$ involves neither. The default is *recompute* (I-11): ξ, μ are reused from the model's sweep (identical at the manifold frame (U, O, P, Q) and the corewise frame (U, P, P, P) of the base point), ν is re-swept through G (1–3% of the two-hole), and

nothing else is stored. (At the manifold frame the corewise state is a cheap T/S transform of the manifold one, $\nu_c = T\nu$, $\eta_c = S\eta$ — F16 — which a deferred "store" variant would use.)

The block-sum oracle. Since $D\Phi(\theta)[\hat{\theta}] = \Phi(Y)$ for the doubled-rank T3 Y that `tv_to_t3` builds from $\hat{\theta}$ at `t3_corewise_frame`(θ), whose blocks are cores of θ, of $\hat{\theta}$ or zero, and Y is affine in θ (its $\hat{\theta}$ blocks fixed),

$$\mathcal{K}_c\hat{\theta} = B^{\mathsf{T}}\,\nabla_Y\langle c,\, A\Phi(Y)\rangle, \tag{15}$$

B^{T} summing the gradient blocks where θ's cores sit. This one-hole-only identity (check C9b, 3.4×10^{-16}) is the reference every two-hole is tested against (`tests/backend/_curvature_oracles.py`); the others are `jax.jvp` of the shipped one-hole and, for the whole Hessian, a dense extrinsic Weingarten oracle (`tests/oracle_sweeps/sweep_exact.py`).

3.3 Two-index jets and the recurrence-only rule

For the derivative kinds every edge variable is a jet $x = (x^{(0)}, \dots, x^{(m)})$ along the sample direction $w + sp$, and the core direction is a *second, independent* index truncated at first order. Leibniz in the core index needs no binomials, $\frac{d}{ds}(fgh) = \dot{f}gh + f\dot{g}h + fg\dot{h}$, so each three-factor jet product of the GN kernels spawns three dotted products with the same order wiring. Layout: *companion arrays* (I-10) — undotted jets $(m{+}1,) + \mathtt{W} + \mathtt{C} + (\cdot)$, dotted ones $(m{+}1,) + \mathtt{W} + \mathtt{K} + \mathtt{C} + (\cdot)$ — never a stacked length-2 axis, which would broadcast every undotted jet over `K`.

Write the down-shift $(\mathsf{D}x)^{(t)} = t\,x^{(t-1)}$ and its adjoint, the up-shift $(\mathsf{U}x)^{(s)} = (s+1)\,x^{(s+1)}$ ($x^{(m+1)} = 0$), so that $\mathsf{D}^r x = r!\binom{t}{r}x^{(t-r)}$ and $(\mathsf{U}^r x)^{(s)} = r!\binom{r+s}{r}x^{(r+s)}$ (`_shift_down`, `_shift_up`). The input jets are affine, $\xi_i = (U_i w_i, U_i p_i)$, $\dot{\xi}_i = (\dot{U}_i w_i, \dot{U}_i p_i)$ (`build_input_jets`). Every jet product is one of four forms:

$$\text{banded (one affine factor } b\text{):}\quad (a \star b)^{(t)} = a^{(t)}b^{(0)} + (\mathsf{D}a)^{(t)}b^{(1)},\quad (c \star^{\mathsf{T}} b)^{(s)} = c^{(s)}b^{(0)} + (\mathsf{U}c)^{(s)}b^{(1)}, \tag{16}$$

$$\text{full (both jets):}\qquad (a * b)^{(t)} = \textstyle\sum_r \frac{a^{(r)}}{r!}(\mathsf{D}^r b)^{(t)},\qquad (a *^{\mathsf{T}} z)^{(s)} = \sum_r \frac{a^{(r)}}{r!}(\mathsf{U}^r z)^{(s)}, \tag{17}$$

the banded ones a two-term recurrence fused into one GEMM (`_shift_down_pair`, `_shift_up_pair`), the full ones an *order scan* over r that carries the shifted factor, one order slice live at a time (`_inverse_factorials` supplies $1/r!$). **The recurrence-only rule** (I-33): every production kernel is one of these forms; a dense binomial tensor $\binom{t}{r}[r+s=t]$ appears only in the `_trs` *test twins* (`tests/_jet_trs_twins.py`, `tests/backend/_curvature_oracles.py`), which the kernels match to $\le 5.8 \times 10^{-16}$ over orders 0–4.

The jet sweeps. With these, the GN adjoint step (`_adj_tilde_step`) is $\tilde{\tau}_i = \tilde{\tau}_{i-1} \star^{\mathsf{T}} G_i(\xi_i) + \mu_{i-1} *^{\mathsf{T}} G_i(\delta\tilde{\eta}_i)$ — a *prop* half (`_adj_prop`) and a *src* half (`_adj_src_order_step`) — that is,

$$\tilde{\tau}_i^{(s)} = \tilde{\tau}_{i-1}^{(s)} G_i(\xi_i^{(0)}) + (s+1)\,\tilde{\tau}_{i-1}^{(s+1)} G_i(\xi_i^{(1)}) + \sum_{r\ge 0}\tbinom{r+s}{r}\,\mu_{i-1}^{(r)} G_i\big(\delta\tilde{\eta}_i^{(r+s)}\big), \tag{18}$$

and its product rule along $(\dot{G}, \dot{\xi}, \dot{\mu}, \dot{\delta\tilde{\eta}})$, with $\dot{\delta\tilde{\eta}}_i = \dot{U}_i c_i$, is the dotted step (`_adj_tilde_tangent_step`; check C10d):

$$\dot{\tilde{\tau}}_i = \underbrace{\dot{\tilde{\tau}}_{i-1} \star^{\mathsf{T}} G_i(\xi_i) + \tilde{\tau}_{i-1} \star^{\mathsf{T}} \big(\dot{G}_i(\xi_i) + G_i(\dot{\xi}_i)\big)}_{\text{prop: } \texttt{_adj_prop_tangent}\text{, two GEMMs}} + \underbrace{\big(\dot{\mu}_{i-1}G_i + \mu_{i-1}\dot{G}_i\big) *^{\mathsf{T}} \delta\tilde{\eta}_i + \mu_{i-1}G_i *^{\mathsf{T}} \dot{\delta\tilde{\eta}}_i}_{\text{src: } \texttt{_adj_tangent_src_order_step}\text{, one scan over } r}, \tag{19}$$

Algorithm 4 Two-hole, forward half: the dotted forward jets along $\hat{\theta} = (\dot{U}, \dot{G})$

Require: $\theta = (U, G)$; $\hat{\theta}$ (stack `K + C`); w_i, p_i (stack `W`); order m; model sweep (ξ, μ)
Ensure: $\dot{\xi}_i, \dot{\mu}_i, \dot{\nu}_i, \dot{\eta}_i$ (orders $0..m$) and the undotted ν_i

$\dot{\xi}_i \leftarrow (\dot{U}_i w_i,\ \dot{U}_i p_i)$ ▷ `build_input_jets`, `compute_xi`
$\nu_{i-1} \leftarrow G_i(\xi_i) \star \nu_i$ ▷ re-swept through G: `compute_nu_jets`
for $i = 1, \dots, d$ **do**
  $\dot{\mu}_i \leftarrow \dot{\mu}_{i-1} \star G_i(\xi_i) + \mu_{i-1} \star \dot{G}_i(\xi_i) + \mu_{i-1} \star G_i(\dot{\xi}_i)$ ▷ `compute_sigma_jets`$(\dot{G}, G, G, \dots)$
end for
for $i = d, \dots, 1$ **do**
  $\dot{\nu}_{i-1} \leftarrow G_i(\xi_i) \star \dot{\nu}_i + \dot{G}_i(\xi_i) \star \nu_i + G_i(\dot{\xi}_i) \star \nu_i$ ▷ `compute_tau_jets`
end for
$\dot{\eta}_i \leftarrow \dot{\mu}_{i-1} G_i * \nu_i + \mu_{i-1} \dot{G}_i * \nu_i + \mu_{i-1} G_i * \dot{\nu}_i$ ▷ `compute_deta_jets` (order scan)

Code: `sampling_derivatives`: the GN tangent forward (T4S Alg. 7 jet-ified) at $Q = O = P = G$; plain kinds `probing.compute_dxi`, `compute_sigma`, `compute_tau`, `compute_deta`.

Algorithm 5 Two-hole, adjoint half: the dotted adjoint sweeps

Require: undotted ξ, μ, ν and adjoint state $\delta\tilde{\eta}, \tilde{\tau}, \tilde{\sigma}$ at θ; dotted $\dot{\xi}, \dot{\mu}, \dot{\nu}$; residual jets c_i
Ensure: $\dot{\delta\tilde{\eta}}_i, \dot{\tilde{\tau}}_i, \dot{\tilde{\sigma}}_i$ and the two read rows $\dot{\delta\tilde{\xi}}_i^{(0,1)}$

$\dot{\delta\tilde{\eta}}_i \leftarrow \dot{U}_i c_i$ ▷ order-diagonal; `compute_deta_tilde_tangent_jets`
for $i = 1, \dots, d$ **do** ▷ carry $\dot{\tilde{\tau}}$; the undotted $\tilde{\tau}_{i-1}$ rides as scan data
  $\dot{\tilde{\tau}}_i \leftarrow$ (19) ▷ `compute_tau_tilde_tangent_jets`, step `_adj_tilde_tangent_step`
end for
$\dot{\tilde{\sigma}} \leftarrow$ the same sweep on the reversed train ▷ `compute_sigma_tilde_tangent_jets`
$\dot{\delta\tilde{\xi}}_i^{(u)} \leftarrow$ the product rule of (20), $u \in \{0, 1\}$ ▷ `compute_dxi_tilde_tangent_jets`

Code: `sampling_derivatives`: `_adj_prop_tangent`, `_adj_tangent_src_order_step`, `_adj_tangent_sweep_scanned`, `_dxi_tilde_tangent_order_step`; plain probe: `probing.compute_deta_tilde_tangent`, `compute_tau_tilde_tangent` (two `_sigma_step` calls per core), `compute_sigma_tilde_tangent`, `compute_dxi_tilde_tangent` (two `compute_deta` calls).

the six dotted terms (K on the carry, the core, ξ; on the edge, the core, $\delta\tilde{\eta}$) paired by shared factor; the src scan's live slice is $\mathtt{WK}r^2$. $\dot{\tilde{\sigma}}$ is the mirror (`tt_reverse`). Since $\delta\tilde{\xi}$ is the adjoint of the *affine* $\delta\xi$, only its rows $u \in \{0, 1\}$ are read (the Tucker assembly pairs them with (w, p)), and only those are formed: with $[\mathsf{U}^0 x] = x$, $[\mathsf{U}^1 x] = \mathsf{U}x$,

$$\delta\tilde{\xi}_i^{(u)} = \sum_j \mu_{i-1}^{(j)} G_i \, [\mathsf{U}^u \tilde{\sigma}_i]^{(j)} + [\mathsf{U}^u \tilde{\tau}_{i-1}]^{(j)} G_i \, \nu_i^{(j)}, \qquad u \in \{0, 1\}, \tag{20}$$

an order scan over j (`compute_dxi_tilde_jets`); its dot has the six terms $d(\mu G)[\mathsf{U}^u\tilde{\sigma}] + \mu G[\mathsf{U}^u\dot{\tilde{\sigma}}] + [\mathsf{U}^u\dot{\tilde{\tau}}]G\nu + [\mathsf{U}^u\tilde{\tau}]d(G\nu)$, $d(\mu G) = \dot{\mu}G + \mu\dot{G}$, $d(G\nu) = \dot{G}\nu + G\dot{\nu}$ (`_dxi_tilde_tangent_order_step`; check C10e).

3.4 The assembly, factored by the product rule

The Tucker block is the derivative of a single order-sum, `W` summed:

$$(\mathcal{K}_c\hat{\theta})_{U_i} = \sum_{\mathtt{W}} \Big(\sum_t \dot{\eta}_i^{(t)} \otimes c_i^{(t)} + \sum_{u\in\{0,1\}} \dot{\tilde{\xi}}_i^{(u)} \otimes w_i^{(u)} \Big), \qquad w^{(0)} = w,\ w^{(1)} = p. \tag{21}$$

The TT block is the derivative of three *trilinear* groups (eq. (14)); the product rule gives nine trilinear terms, each a `W`-summed outer product of the size of a core. Forming each group's *pair intermediate* and its dotted twin once, before the `W` sum, replaces the nine large contractions by fewer: four for the plain probe, two plus two per lag step for the jets, two (from three) for apply / entries (below). (F33 measured the planned six-final factored jet form 1.4–2.0× faster on that term than the nine-term one, equal to 3×10^{-15}.)

Plain probe (`probing.assemble_tt_tangent_variations`). Group by the left-bond factor: $\delta\tilde{G}_i = \sum_{\mathtt{W}} \mu_{i-1} \cdot E_1 + \tilde{\tau}_{i-1} \cdot E_2$ with the pairs (over the mode and right-bond legs)

$$E_1 = \xi_i \otimes \tilde{\sigma}_i + \delta\tilde{\eta}_i \otimes \nu_i, \qquad E_2 = \xi_i \otimes \nu_i, \qquad (\mathcal{K}_c\hat{\theta})_{G_i} = \sum_{\mathtt{W}} \dot{\mu}_{i-1} \cdot E_1 + \mu_{i-1} \cdot \dot{E}_1 + \dot{\tilde{\tau}}_{i-1} \cdot E_2 + \tilde{\tau}_{i-1} \cdot \dot{E}_2, \tag{22}$$

$\dot{E}_1 = \dot{\xi} \otimes \tilde{\sigma} + \xi \otimes \dot{\tilde{\sigma}} + \delta\dot{\tilde{\eta}} \otimes \nu + \delta\tilde{\eta} \otimes \dot{\nu}$, $\dot{E}_2 = \dot{\xi} \otimes \nu + \xi \otimes \dot{\nu}$: four `W`-summed finals (check C11c). Any pairing of the three groups into two intermediates gives four; slice S3's dense form paired the $\tilde{\tau}$ and $\delta\tilde{\eta}$ groups through their common ν.

Jets (`assemble_tt_tangent_variation_jets`). The $\tilde{\sigma}$ and $\tilde{\tau}$ groups share the *affine* ξ, so they are banded and add into one pair; the $\delta\tilde{\eta}$ group (both μ, ν full) is a *lag scan*. The GN assembly (`assemble_tt_variation_jets`) is

$$\delta\tilde{G}_i = \sum_{\mathtt{W}} \Big(\sum_{k\in\{0,1\}} \Pi_{i,k} \otimes_2 \xi_i^{(k)} + \sum_r \mu_{i-1}^{(r)} \otimes F_i^{(r)} \Big), \quad \Pi_{i,k} = \sum_r \mu_{i-1}^{(r)} \otimes [\mathsf{U}^k\tilde{\sigma}_i]^{(r)} + \sum_s [\mathsf{U}^k\tilde{\tau}_{i-1}]^{(s)} \otimes \nu_i^{(s)}, \tag{23}$$

with the fold $F_i^{(r)} = \sum_s \binom{r+s}{r} \delta\tilde{\eta}_i^{(r+s)} \otimes \nu_i^{(s)} = \sum_s \frac{1}{r!} (\mathsf{U}^r\delta\tilde{\eta}_i)^{(s)} \otimes \nu_i^{(s)}$ (`_tt_det_lag_step`, which carries $\mathsf{U}^r\delta\tilde{\eta}$). Its derivative (`_tt_tangent_variation_jets_step`, `_tt_tangent_det_lag_step`) is

$$(\mathcal{K}_c\hat{\theta})_{G_i} = \sum_{\mathtt{W}} \Big(\sum_k \dot{\Pi}_{i,k} \otimes_2 \xi_i^{(k)} + \Pi_{i,k} \otimes_2 \dot{\xi}_i^{(k)} + \sum_r \dot{\mu}_{i-1}^{(r)} \otimes F_i^{(r)} + \mu_{i-1}^{(r)} \otimes \dot{F}_i^{(r)} \Big), \tag{24}$$

$\dot{\Pi}_{i,k}$ the four-term product rule of $\Pi_{i,k}$ ($\mathtt{K}$ on $\dot{\mu}$, $\dot{\tilde{\sigma}}$, $\dot{\tilde{\tau}}$, $\dot{\nu}$), $\dot{F}^{(r)}$ the two-term one: two banded finals plus two per lag step (check C11a: equal to the nine-term dense binomial form to 6.3×10^{-16}). Apply/entries have one group, so $3 \to 2$ finals: $(\mathcal{K}_c\hat{\theta})_{G_i} = \sum_{\mathtt{W}} \dot{\mu}_{i-1} \cdot E + \mu_{i-1} \cdot \dot{E}$, $E = \xi_i \otimes \hat{\sigma}_i$. The probe-derivative assemblies are `W`-chunked through `_wchunked_sum` (per-chunk kernels `_tt_tangent_chunk_summed`, `_tucker_tangent_chunk_summed`); the other kinds' transposes have no chunked path, so neither do their two-holes.

Apply and entries. Their two-hole differentiates the seeded sweep: $\hat{\sigma}$ is re-swept through G from the seed c, and its derivative starts from *zero* (the seed is fixed), $\dot{\hat{\sigma}}_{i-1} = G_i(\xi_i) \star^{\mathsf{T}} \dot{\hat{\sigma}}_i + (\dot{G}_i(\xi_i) + G_i(\dot{\xi}_i)) \star^{\mathsf{T}} \hat{\sigma}_i$ (`_adj_prop_tangent` again); then $\delta\dot{\hat{\xi}} = d(\mu G\hat{\sigma})$ in its two read rows, and the two-final assembly. Entries is apply with one-hot vectors (`_entries_ambient_jets`, fiber-sliced ξ). At order 0 every jet kernel reduces to the plain kind's (check C9d).

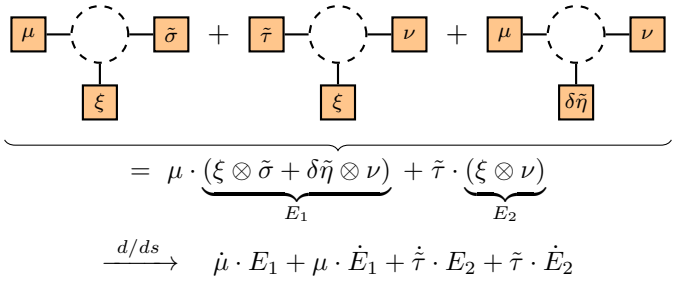


Figure 3: The TT gradient block as three trilinear outer products around the hole at core i (left bond, mode leg, right bond), and its factored derivative: the pairs E_1, E_2 (and their dotted twins) are formed per sample, then four `W`-summed contractions with the left-bond factor produce the core-sized result, instead of nine.

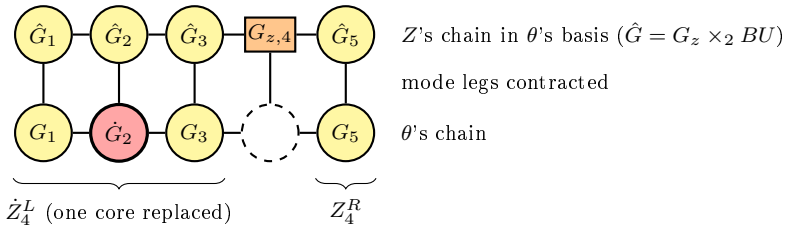


Figure 4: The T3-to-T3 two-hole as a ladder: Z's re-based TT cores on top, θ's below. The hole at core 4 is the gradient block; the bottom-left $\dot{G}_2$ makes $\dot{Z}_4^L$ one term of the tangent zipper. Core 4's own top rung is $G_{z,4}$ itself, contracted with BU_4 (or $\dot{BU}_4$) on the mode leg.

3.5 The T3-to-T3 two-hole

$\mathcal{K}_{\text{grad}\, f}$ is (5) for $Z = \text{grad}\, f$ given as a T3 (B_z, G_z) of doubled rank. The one-hole (`t3_inner_product_corewise_transpose`) re-expresses Z's TT cores in θ's Tucker basis and zips them against θ's chain:

$$BU_i = B_{z,i} U_i^{\mathsf{T}}, \quad \hat{G}_i = G_{z,i} \times_2 BU_i, \quad Z^L = \text{zip}_{\rightarrow}(\hat{G}, G), \quad Z^R = \text{zip}_{\leftarrow}(\hat{G}, G), \quad \text{env}_i = Z_i^L\, G_{z,i}\, Z_i^R, \tag{25}$$

with gradient blocks $\text{env}_i \times_2 BU_i$ (TT) and $\langle \text{env}_i,\, G_i \rangle^{\mathsf{T}} B_{z,i}$ (Tucker). The two-hole adds the "one core replaced" companions: $\dot{BU}_i = B_{z,i} \dot{U}_i^{\mathsf{T}}$, $\dot{\hat{G}}_i$, the *tangent zippers* $\dot{Z}_{i+1}^L = \dot{Z}_i^L A_i B_i + Z_i^L \dot{A}_i B_i + Z_i^L A_i \dot{B}_i$ (for the chains $A = \hat{G}$, $B = G$; `tt_zipper_tangent_left_to_right`, step `_tt_zipper_tangent_step`), and the product rule

$$\begin{aligned} \dot{\text{env}}_i &= \dot{Z}_i^L G_{z,i} Z_i^R + Z_i^L G_{z,i} \dot{Z}_i^R, \\ (\mathcal{K}_Z \dot{\theta})_{G_i} = \dot{\text{env}}_i \times_2 BU_i + \text{env}_i \times_2 \dot{BU}_i, &\qquad (\mathcal{K}_Z \dot{\theta})_{U_i} = \big(\langle \dot{\text{env}}_i,\, G_i \rangle + \langle \text{env}_i,\, \dot{G}_i \rangle\big)^{\mathsf{T}} B_{z,i}. \end{aligned} \tag{26}$$

Its cost is independent of the number of samples. (The one-hole moved from `tv_operations` to `t3_linalg`; its frame kernel, at a manifold frame and gauged, is `tv_project_t3_onto_tangent_space`, bit-identical.)

3.6 The uniform masking rule

On the uniform (packed) layout a second derivative multiplies a direction's *padding* into real slots: the masks are load-bearing, not hygiene (an unmasked $\hat{\theta}$ corrupts the real block by 2.6×10^3; F15,

Algorithm 6 The probe-derivative two-hole $\mathcal{K}_c\hat{\theta}$ (driver)

Require: $c_i = \omega^2 \odot r_i$ (jets, $\mathtt{W}+\mathtt{C}$); w, p; $\theta = (U, G)$; $\hat{\theta}$ ($\mathtt{K}+\mathtt{C}$); order m; model sweep (ξ, μ)
Ensure: $\mathcal{K}_c\hat{\theta}$, raw core variations ($\mathtt{K}+\mathtt{C}$, $\mathtt{W}$ summed)
1: undotted: ν re-swept through G; $\delta\tilde{\eta} = Uc$; $\tilde{\tau}$, $\tilde{\sigma}$ ▷ GN kernels, recompute (I-11)
2: dotted forward $\dot{\xi}, \dot{\mu}, \dot{\nu}, \dot{\eta}$ ▷ Algorithm 4
3: dotted adjoint $\delta\dot{\tilde{\eta}}, \dot{\tilde{\tau}}, \dot{\tilde{\sigma}}, \delta\tilde{\xi}^{(0,1)}$ ▷ Algorithm 5
4: Tucker block (21); TT block (24), $\mathtt{W}$-chunked ▷ Section 3.4
Code: `sampling_derivatives.t3_probe_corewise_derivatives_hessian[_from_sweep]`; step 1 `compute_nu_jets`, `compute_deta_tilde_jets`, `compute_tau_tilde_jets`, `compute_sigma_tilde_jets`; step 4 `assemble_tucker_tangent_variation_jets`, `assemble_tt_tangent_variation_jets`; the other kinds: `probing.t3_probe_corewise_hessian[_from_sweep]`, `apply.t3_apply_corewise_hessian[_from_sweep]` (`_apply_corewise_hessian_adjoint`), `entries.t3_entries_corewise_hessian[_from_sweep]`, `sampling_derivatives.t3_apply_corewise_derivatives_hessian[_from_sweep]`, `t3_entries_corewise_derivatives_hessian[_from_sweep]` (`_apply_derivatives_hessian_from_jets`, `compute_sigma_hat_tangent_jets` with step `_sigma_hat_tangent_jets_step`); the hook `SamplingKind.corewise_hessian` (the required sixth operation) in `fitting.py`, `uniform_fitting.py`.

F18). Every uniform `corewise_hessian` therefore masks θ *and* $\hat{\theta}$ with the masks of the **frame it is handed** — `frame_data[5]`'s up and left masks (`uniform_fitting._frame_theta`), not the kind's `self.masks`, which is a strict superset at TT slack (F23) — runs the mask-free polymorphic kernel, and re-masks the output (`ut3_sampling._masked_point_and_directions`, `_remasked_core_gradient`; `ut3_masking.ut3_apply_masks`). "Garbage" here means *finite* garbage (NaN leaks through any mask that multiplies). The uniform lift factors clip each block at its real dimensions; padded singular values are exactly 0.

4 Shared (tied) Tucker factors

On the shared set (SF-T3, after the SF-ETT format of [5]) the point is $\Phi(\mathsf{T}\theta_{sh})$, T copying each group's factor to its modes, so the Hessian is the same formula through a *tied* lift (`sharing.fv_tied_lift_factors`). A tied coordinate $\delta U_i = S_i^{\mathsf{T}}\dot{U}_g$ (`docs/shared_t3_math.tex`, Prop. "tied tangent space is a tilted subspace"; that note writes the factors $N \times n$, so its $C_i = \dot{U}S_i$ is this relation transposed) has one raw factor direction per group, recovered from all the group's coordinates at once against the stacked $M_g = [S_{i_1}^{\mathsf{T}}; \ldots; S_{i_k}^{\mathsf{T}}]$ and its SVD $M_g = U_M \operatorname{diag}(s_g)\, V_M^{\mathsf{T}}$ (the companion `SharedFrameData` that the tied projection already uses):

$$\dot{U}_g = M_g^{+\varepsilon} \begin{bmatrix} \delta U_{i_1} \\ \vdots \\ \delta U_{i_k} \end{bmatrix} = V_M f_\varepsilon(s_g) U_M^{\mathsf{T}} [\delta U_i]_{i\in g}, \qquad \mathcal{L}^{\mathsf{T}} : \; [\delta U_i]_{i\in g} = (M_g^{+\varepsilon})^{\mathsf{T}} \textstyle\sum_{i\in g} Y_i^U, \tag{27}$$

the same $\dot{U}_g$ at every group mode; the adjoint sums the group's copies (T^{T}) before the solve's adjoint. Singleton modes and every TT slot lift as unshared. The normal part is taken against the *tied* gradient (the geometry's tied `project`), and the outer Π_{sh} of (10) ties the result.

Algorithm 7 The T3-to-T3 two-hole $\mathcal{K}_Z\hat{\theta}$

Require: $Z = (B_z, G_z)$ (tails squashed); $\theta = (U, G)$ (C); $\hat{\theta}$ (K + C)
Ensure: $\mathcal{K}_Z\hat{\theta}$

1: $BU, \hat{G}, Z^L, Z^R$, env by (25) ▷ per frame: `t3_precompute_inner_product_sweep`
2: $\dot{BU}_i \leftarrow B_{z,i}\dot{U}_i^{\mathsf{T}}$; $\dot{\hat{G}}_i \leftarrow G_{z,i} \times_2 \dot{BU}_i$
3: $\dot{Z}^L \leftarrow$ `tt_zipper_tangent_left_to_right`$(\hat{G}, G, \dot{\hat{G}}, \dot{G}; Z^L)$; $\dot{Z}^R$ likewise from the right
4: blocks by (26)

Code: `t3_linalg.t3_inner_product_corewise_hessian[_from_sweep]`, `t3_linalg.t3_inner_product_corewise_transpose`, `tt_operations.tt_zipper_tangent_left_to_right`, `tt_zipper_tangent_right_to_left`; called by `ManifoldGeometryOps.tangential_curvature`; uniform: `ut3_linalg.ut3_precompute_inner_product_sweep`, `ut3_inner_product_corewise_hessian[_from_sweep]`.

Algorithm 8 The tied lift and its adjoint (one nontrivial group g)

Require: companion $(U_M, s_g, V_M,$ row splits) of M_g; ε; coordinates $[\delta U_i]_{i\in g}$, or raw $[Y_i^U]_{i\in g}$

1: $f \leftarrow f_\varepsilon(s_g)$, clip rcond $=$ dtype $\varepsilon \cdot \max(\sum_{i\in g} n_i^D, n_g)$ ▷ `_tied_filter`
2: lift: $\dot{U}_g \leftarrow V_M \operatorname{diag}(f)\, U_M^{\mathsf{T}}\, [\delta U_i]_{i\in g}$; $\dot{U}_i \leftarrow \dot{U}_g$ for $i \in g$ ▷ `_tied_filtered_solve`
3: adjoint: $Y_g \leftarrow \sum_{i\in g} Y_i^U$; $[\delta U_i] \leftarrow U_M \operatorname{diag}(f)\, V_M^{\mathsf{T}} Y_g$, split by rows ▷ its transpose

Code: `sharing.fv_tied_lift_factors` (fills `LiftFactors.shared_data`, `tied_filters`), `sharing._tied_filter`, `_tied_filtered_solve[_transpose]` (with `eps=None` bit-identical to the tied projection's solve), dispatched inside `tv_corewise_lift[_transpose]`; uniform: `ufv_operations.ufv_tied_lift_factors`.

- s_g **replaces the group modes' own spectra.** The tied lift inverts M_g, never a group mode's S_i^{T}, which can be rank-deficient at a perfectly regular shared point (a group rank above a member's ceiling $r_{i-1}r_i$). So `lift_sigma_min` $= \min(\sigma(T_i),\ \sigma(S_i)$ for untied i, $s_g)$ (`geometry._lift_sigma_min`). A per-mode lift represents the same tangent but not a tied direction; its quadratic form is off by 5.6–19 at such points (the tests use these *group-ceiling* structures).
- **Padded restarts.** A zero-padded shared restart's new s_g levels are zero and filter to exactly 0: the gated tied channel gets no curvature, as it gets no first-order motion; the TT slots carry the escape. On the uniform layout the group filter is additionally zeroed on the padded s_g levels by the frame's group rank mask and clipped at the group's real dimensions (`ufv_operations.ufv_tied_lift_factors`).
- The shared *corewise* geometry needs no lift: its exact Hessian is the per-group mean (its tied `project`) of the plain one. The shared retraction's second order is measured (F14), not proven.

5 The canonical zero-point frame and the Newton reference

The default Newton reference (`g0_reference='zero'` on the manifolds) is $\|\Pi\mathcal{J}^{\mathsf{T}}(\omega^2 \odot r_0)\|$, $r_0 = A \cdot 0 - y$, at the zero tensor of x_0's ranks: the relative stop $\|g\| \le$ `gtol_rel` $\cdot\, \|g_0\|$ and the forcing

term $\eta = \min(\frac{1}{2}, (\|g\|/\|g_0\|)^{\texttt{cg_forcing_power}})$ are measured against it. At $X = 0$ every orthonormal completion is a valid frame and the gradient norm depends on the choice, so the library fixes one (I-29):

Algorithm 9 The canonical zero-point frame

Require: a frame's *structure* (core shapes; unstacked); sharing groups (singletons their own); seed $= 0$

1: **for** each group g with key $k_g = \min g$ **do**
2: $G \leftarrow$ `RandomState([seed,k_g]).standard_normal((n_g,N_g))` ▷ keyed legacy stream
3: $Q, R \leftarrow \mathrm{qr}(G^{\mathsf{T}})$ (reduced); $U_g \leftarrow (Q\,\mathrm{sign}(\mathrm{diag}R))^{\mathsf{T}}$, replicated over g ▷ sign-fixed Haar rows
4: **end for**
5: $O_i, P_i, Q_i \leftarrow$ identity prefixes (orthonormal), $P_d = 0$, $Q_1 = 0$ ▷ base point $= 0$

Code: `fv_operations.fv_zero_point_frame` (`_haar_rows`, `_identity_prefix`, `ZERO_FRAME_SEED`); uniform `ufv_operations.ufv_zero_point_frame` (unpack, build, pack); geometry member `zero_point_frame`; the reference: `optimizers._resolve_g0_reference`, `_zero_reference_gnorm`.

Three properties hold by construction: *rank independence* (each group draws from its own keyed stream, so a factor never depends on another mode's rank or size — bit-identical); *nesting* (QR column j depends only on draw rows $\le j$, so level r's factor is exactly the leading-row prefix of level $r{+}1$'s, and the references along a continuation ladder are taken on nested subspaces); *layer independence* (the uniform twin builds the same real blocks, $\le 3.9 \times 10^{-16}$; the 44 ragged/uniform optimizer pairs agree to 3.5×10^{-12}, F37). Haar only for U: the Tucker factor is the only part of the frame that meets the data, and any coordinate-aligned completion gives a reference of exactly 0 on data with no energy on the leading coordinates (F32; Haar seeds 0–5 gave 9.5–24 there). The reference is a *convention* — a deterministic data-gradient scale at these ranks, good to an $O(1)$ factor (another seed or a mode rotation moved it by up to $\sim 1.7\times$) — harmless for a relative stop. The corewise geometries have none (zero cores are a critical point): their default is the warm start. A vanishing zero reference falls back to the warm start with a warning (default) or raises (explicit).

6 The trust region

The exact Hessian is indefinite away from a minimizer, so `hessian='exact'` defaults to `step_control='trust_region'`: Steihaug–Toint CG [7, 8] inside $\|p\| \le \Delta$ and one trial retraction, accepted on the ratio ρ of actual to predicted change. This is what makes the exact model's ρ meaningful: with a second-order retraction the predicted and actual changes agree to $O(\|p\|^3)$.

One CG body for both step controls (`_cg_body`, I-5). The state carries the radius and a boundary flag; every branch is an `xnp.where`, so the same body drives the eager loop and `lax.while_loop`. With $\alpha = r^{\mathsf{T}}r/d^{\mathsf{T}}\mathcal{H}d$: if $d^{\mathsf{T}}\mathcal{H}d > 0$ and $\|p + \alpha d\| \le \Delta$ it is a plain CG step; if $\|p + \alpha d\| > \Delta$, or $d^{\mathsf{T}}\mathcal{H}d \le 0$ with Δ finite, it steps to the boundary,

$$p \leftarrow p + \tau d, \qquad \tau = \frac{-p^{\mathsf{T}}d + \sqrt{(p^{\mathsf{T}}d)^2 + \|d\|^2(\Delta^2 - \|p\|^2)}}{\|d\|^2} \quad (\|p + \tau d\| = \Delta,\ \tau \ge 0), \tag{28}$$

and stops. With $\Delta = \infty$ (line search) nonpositive curvature takes a zero step and stops: the pre-trust-region truncated CG, bit for bit on eager numpy. No ∞ ever enters arithmetic.

Algorithm 10 Newton–CG with the trust region (one row per trial)

Require: x_0; references $\|g_0\|$ (Section 5); forcing power κ (default $\frac{1}{2}$; 1 for a q-quadratic tail)

build the local model at x; g, f ▷ `Problem.local_model(hessian='exact')`
loop
  if $\|g\| \le$ `gtol_rel` $\|g_0\|$ **then return** x (`'gradient'`)
  end if
  $\eta \leftarrow \min(\frac{1}{2}, (\|g\|/\|g_0\|_{cg})^{\kappa})$ ▷ forcing term
  first step: $\Delta_0 \leftarrow \|g\|^3 / g^{\mathsf{T}} \mathcal{H}_{GN} g$ (or $\|g\|$), $\Delta_{\max} \leftarrow 10^4 \Delta_0$ ▷ `_initial_radius`
  $p \leftarrow$ Steihaug CG on $\mathcal{H}p = -g$ to $\eta\|g\|$ inside Δ ▷ `_cg_solve`
  if $g^{\mathsf{T}} p \ge 0$: $p \leftarrow -g \min(1, \Delta/\|g\|)$ (a boundary step if scaled)
  pred $\leftarrow g^{\mathsf{T}} p + \frac{1}{2}(p^{\mathsf{T}} \mathcal{H}_{GN} p + p^{\mathsf{T}} \mathcal{L}^{\mathsf{T}} \mathcal{K}_N \mathcal{L} p)$ ▷ one two-hole
  $\rho \leftarrow (f(R_x(p)) - f)/\text{pred}$ if pred < 0, else NaN
  if $\rho < \frac{1}{4}$ or NaN: $\Delta \leftarrow \frac{1}{4}\|p\|$; **elif** $\rho > \frac{3}{4}$ and boundary: $\Delta \leftarrow \min(2\Delta, \Delta_{\max})$
  if $\rho > 0.1$ **then** accept: $x \leftarrow R_x(p)$, rebuild the model
  else keep x and its model
    stop `'objective_floor'` if $|\text{pred}| \le 10\,\epsilon_{\text{res}}(\|\omega \odot y\|\,\|\omega \odot r\| + |f|)$
    stop `'trust_region_floor'` if $\Delta < 10^{-12}\Delta_0$ (checked second)
  end if
end loop

Code: `optimizers.newton_cg`; CG `_cg_solve`, `_cg_core`, `_cg_body`; `LocalModel.gn_quadratic`, `curvature_quadratic` (constants `_TR_ACCEPT`, `_TR_SHRINK_BELOW`, `_TR_SHRINK`, `_TR_EXPAND_ABOVE`, `_TR_EXPAND`, `_TR_RADIUS_MAX_REL`, `_TR_RESOLUTION_FACTOR`, `_TR_RADIUS_FLOOR_REL`); ϵ_{res} = `_resolution_eps` (host-side, dtype-only); per-row `NewtonInfo`.

The floors. They are checked only after a *rejection* (an accepted step never stops the run). ϵ_{res} = max `np.finfo(dtype).eps` over the residual's floating leaves, read from dtypes only (no device sync), so a float32 fit is judged at float32's resolution. `'objective_floor'` is not a failure: the model change is below the objective's floating-point resolution, so the attainable accuracy is reached. A rejected trial keeps x and its local model and emits its own history row; Δ_0 is the Gauss–Newton Cauchy length whatever the Hessian. Measured (seeded probe-derivative restart, F19): Gauss–Newton with Armijo 39 steps / 130 Hessian-applies, exact with the trust region 8 / 23 at power 1.0, 10 / 28 at 0.5, 0 rejected; the q-quadratic tail $e_{k+1} \le 0.92\, e_k^2$ at power 1.0 (F22).

7 Verification

Every displayed identity was checked against the library at `f739085` by a scratch script (numpy float64; jax float64 for the Lagrangian; complex-step derivatives, which are exact, for the two-holes); Table 1 lists the relative residuals. At C8 the point was far from a minimizer: the exact quadratic form was -5449.96 against the Gauss–Newton 30.33, the retraction difference quotient -5449.94. The living guarantees are `tests/backend/test_curvature.py`, `tests/backend/test_uniform_curvature.py`, the exact oracle sweep (720 always-on cases, max 2.5×10^{-14}; 8000 slow-tier cases, max 2.6×10^{-11}) and the dispatch tests' `TestCurvatureJvpOracle` (`jax.jvp` of each one-hole, $\le 10^{-12}$). A frontend finite-difference check of the Hessian (`MANIFOLD.norm(H_fd-hessian(p))`) passes the safe-mode gauge check, which is scaled by the operands (`tv_gauge_residual(frame, variations, scale)`, I-38). Measurements: `docs/contributor/verification_records.md`; design record: `docs/contributor/curvature_internals.md`.

	check	residual
C1	(7): $H_i = O_i \times_2 S_i^\mathsf{T}$; $P_{>i} = T_i Q_{>i}$; spectra; $\|X\|^2 = \|S_1^\mathsf{T}\|_F^2$	$\leq 8.4{\times}10^{-16}$
C2	Prop. 2: $D\Phi[\mathcal{L}v] = \mathcal{E}v$ on minimal / Tucker-slack / TT-slack frames	$\leq 2.4{\times}10^{-15}$
C3–4	$\mathcal{L}_\varepsilon^\mathsf{T}$ is the adjoint at $\varepsilon = 10^{-2}\|X\|$; $\max f_\varepsilon = 1/(2\varepsilon)$; $M^{+\varepsilon} = (M^\mathsf{T}M + \varepsilon^2 I)^{-1}M^\mathsf{T}$	$\leq 1.9{\times}10^{-15}$
C5	$\operatorname{grad} f = P_X \nabla f$; N is normal	$\leq 9.9{\times}10^{-16}$
C6	$\langle N, D^2\Phi[\hat\theta, \hat\theta']\rangle = \langle \hat\theta', \mathcal{K}_N\hat\theta\rangle$ (dense double replacement); a vertical $\hat\theta_v$ has $\mathcal{K}_N\hat\theta_v = 0$ and does not change the form, while $\|\mathcal{K}_{A^\mathsf{T}r}\hat\theta_v\| \approx 10^4$	$\leq 1.2{\times}10^{-13}$
C7	(10) $= \Pi\mathcal{L}^\mathsf{T}\nabla_\theta^2\Lambda\mathcal{L}\Pi$ (`jax.jvp` of `jax.grad`); $\nabla_\theta\Lambda = 0$; level 1 $= \Pi\mathcal{L}^\mathsf{T}\nabla_\theta^2(f\circ\Phi)\mathcal{L}\Pi$	$\leq 4.5{\times}10^{-14}$
C8	$\langle p, \mathcal{H}p\rangle$ vs $\frac{d^2}{dt^2}f(R(tp))$ (Richardson; level 2) / vs $\frac{d^2}{dt^2}f(\Phi(\theta + t\mathcal{L}p))$ (level 1)	$3.7{\times}10^{-6}$ / $2.5{\times}10^{-10}$
C9	the two-holes (probe derivatives, apply derivatives, T3–T3) = complex-step d/ds of their one-holes; the block sum (15); symmetry; order 0 = plain probe	$\leq 4.6{\times}10^{-16}$
C10	the written recurrences $\dot\mu$, (18), (19), $\delta\dot{\tilde{\xi}}$ vs the kernels	$\leq 4.3{\times}10^{-16}$
C11	the factored assemblies (22), (24) = the nine-term binomial product rule	$\leq 6.3{\times}10^{-16}$
C12	tied lift (27): $D\Phi[\mathcal{L}v] = \mathcal{E}v$ at a group-ceiling point; $= M_g^+[\delta U_i]$; adjoint	$\leq 5.1{\times}10^{-16}$
C13	zero frame: nested, rank-independent, base point 0, the recipe by hand (orthonormal to $6.7{\times}10^{-16}$)	0
C14	$\mathcal{H}$ = GN + `curvature_hvp`; quadratic split; exact gradient ≡ GN gradient; default ε	$\leq 4.4{\times}10^{-16}$

Table 1: The equation checks (scratch script against `f739085`; relative residuals).

Costs. An exact Hessian-apply costs 2.8–3.9× a Gauss–Newton one (numpy CPU, six kinds), with a memory peak about twice the GN transpose's; the uniform exact CG compiles once per run.

References

[1] N. Alger, B. Christierson, P. Chen, O. Ghattas, *Tucker Tensor Train Taylor Series*, arXiv:2603.21141 (2026); App. A for the frame, gauge, tangent space and T3-SVD; §6 for probing.

[2] P.-A. Absil, R. Mahony, J. Trumpf, *An extrinsic look at the Riemannian Hessian*, GSI 2013.

[3] M. Psenka, N. Boumal, *Second-order optimization for tensors with fixed tensor-train rank*, arXiv:2011.13395 (2020) — the TT Weingarten map with R_k^{-1}, the analogue of T_i^+.

[4] A. Novikov, M. Rakhuba, I. Oseledets, *Automatic differentiation for Riemannian optimization on low-rank matrix and tensor-train manifolds*, SIAM J. Sci. Comput. 44(2), 2022 (arXiv:2103.14974).

[5] A. Molozhavenko, M. Rakhuba, *Optimization on the extended tensor-train manifold with shared factors*, Comput. Appl. Math. 45:221 (2026); arXiv:2508.20928 (2025).

[6] P.-A. Absil, J. Malick, *Projection-like retractions on matrix manifolds*, SIAM J. Optim. 22(1), 2012.

[7] T. Steihaug, *The conjugate gradient method and trust regions in large scale optimization*, SIAM J. Numer. Anal. 20(3), 1983.

[8] J. Nocedal, S. Wright, *Numerical Optimization*, 2nd ed., Springer 2006, Alg. 4.1 and 7.2.